CHECK WHAT YOU LEARNT

Check Yourself - Unit 1

A. Choose the correct answer

1. Electricity flows only when the circuit is
   (a) broken (b) hot (c) a complete loop (d) made of plastic
2. The long leg of an LED connects towards
   (a) GND (b) the minus side (c) the plus side (d) either side
3. In the middle of a breadboard, holes are joined in
   (a) columns of five (b) rows of ten (c) no pattern (d) pairs
4. A signal that can be any value in a range is called
   (a) digital (b) analog (c) electric (d) binary
5. An Arduino pin can safely supply about
   (a) 20 milliamps (b) 2 amps (c) 20 amps (d) no current
6. The difference between a measured value and the true value is called
   (a) current (b) error (c) voltage (d) resistance

B. Fill in the blanks

1 The push that moves electricity along is called _____.

2 Analog pins on the Arduino Uno are labelled _____ to _____.

3 A motor must never be connected directly to an _____ pin.

4 The battery pack must be _____ before any rewiring.

C. Answer in one or two lines

1 Name one machine at home that follows Sense - Think - Act, and explain each step.

2 Why does accuracy matter far more once a machine starts to move?

3 Your LED does not glow. Write the first three things you would check, in order.

4 Explain why a resistor is needed beside an LED.

UNIT 2

The Arduino IDE

Sessions 3, 4 and 5

Three skills separate this year's work from simple blinking lights: timing something in microseconds, doing arithmetic on the result, and wrapping the whole job inside an instruction you wrote yourself. This unit teaches all three.

By the end of this unit you will be able to:

- Upload a program and use the Serial Monitor to watch live values.
- Use pulseIn() to measure how long a pulse lasts.
- Write your own function and call it from loop().
- Explain why a motor needs a driver and what an H-bridge does.

SESSION 3 — one class

The Board, the Software and the First Upload

Today you will

- Learn the parts of the Arduino Uno board.
- Set the board and port and upload Blink.
- Prove your whole setup works before any project begins.

The Arduino Uno is a small computer with no screen and no keyboard. It runs one program at a time - the one you give it - keeps it in memory when the power is removed, and starts running again the instant power returns. That is why your finished robot still works when you unplug it from the laptop and run it from a battery.

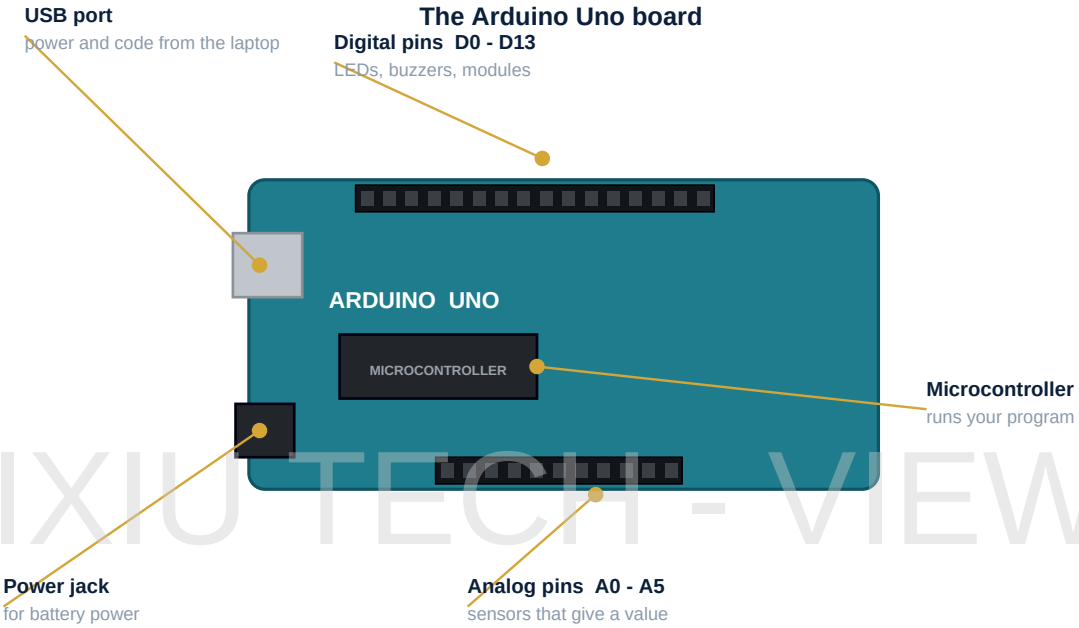

Figure 2.1 - Know your board before you wire anything to it

Group of pins	What is special about them	Used in this book for
D0 to D13	Plain on-or-off inputs and outputs	the ultrasonic sensor, LEDs, the buzzer
~3, ~5, ~6, ~9, ~10, ~11	Can produce a varying output, not just on or off	the servo, and motor speed control
A0 to A5	Measure a voltage and report 0 to 1023	extension work only, this year
5V and GND	Power out to your modules	everything, always

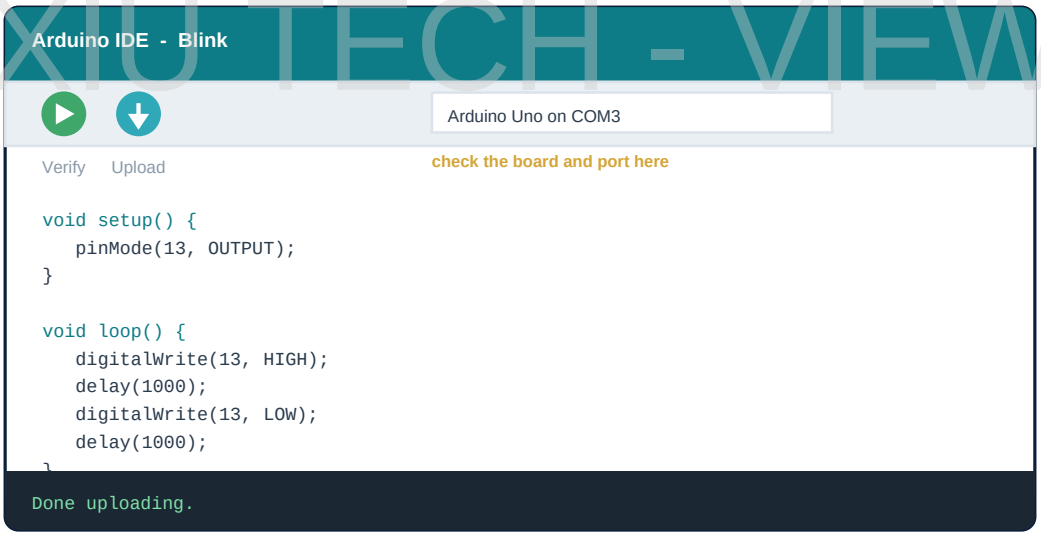

Figure 2.2 - Verify checks your code; Upload sends it to the board

Watch Out

Before your first upload, check **Tools > Board** says Arduino Uno and **Tools > Port** has a port selected. When an upload fails, this is the first thing to check, every single time.

Blink.ino - the test that proves everything works

```
int led = 13;

void setup() {
  pinMode(led, OUTPUT);     // this pin will send power out
}

void loop() {
  digitalWrite(led, HIGH);  // switch it on
  delay(1000);              // wait one second
  digitalWrite(led, LOW);   // switch it off
  delay(1000);
}
```


SESSION 4 — one class

Numbers, Timing and the Serial Monitor

Today you will

- Learn the variable types you will need this year.
- Meet microseconds and the pulseIn() instruction.
- Print live values to the Serial Monitor.

A **variable** is a named box that holds a number. Which type of box you choose matters more this year than it ever has, because the numbers coming back from an ultrasonic sensor can be surprisingly large.

Type	Holds	Biggest value it can store	Use it for
int	A whole number	about 32,767	distances in centimetres
long	A bigger whole number	over 2,000,000,000	times in microseconds
float	A number with a decimal point	very large	heights, averages

Type	Holds	Biggest value it can store	Use it for
boolean	Only true or false	-	flags such as objectSeen

Watch Out

An echo from an object two metres away takes roughly 11,600 microseconds to return - that fits in an int. But if the sensor sees nothing at all it waits far longer, and the number overflows an int and turns negative. That is why the timing value in this book is always stored in a **long**.

4.1 Time, in Slices of a Millionth

You already know `delay(1000)` waits one second, because it counts in **milliseconds** - thousandths of a second. Sound travels far too fast for that to be useful here. Over one millisecond, sound covers about 34 centimetres, so a millisecond-accurate clock could not tell 10 cm from 40 cm.

So the Arduino offers a finer ruler: the **microsecond**, one millionth of a second. Sound travels about 0.034 centimetres in that time - roughly a third of a millimetre. Now you can measure properly. Two instructions matter: `delayMicroseconds()` waits a very short time, and `pulseIn(pin, HIGH)` waits for a pin to go HIGH and reports *how many microseconds it stayed HIGH*.

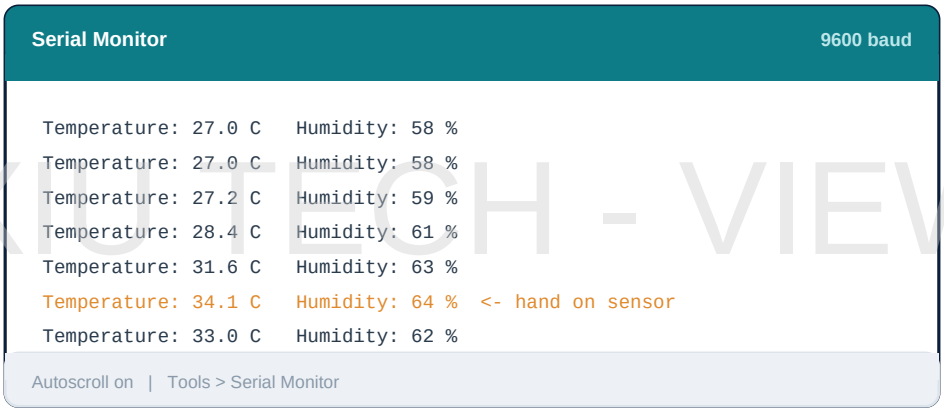

Figure 2.3 - The Serial Monitor is your window into the machine's mind

Try This

Add `Serial.begin(9600);` to setup() and `Serial.println(millis());` to loop() with a `delay(500);` after it. Open the Serial Monitor and watch. `millis()` reports how long the board has been running. Press the reset button and watch the count start again from zero.

SESSION 5 — one class

Loops, Functions and Motors

Today you will

- Use a for loop to repeat something a set number of times.
- Write your own function and call it.
- Understand why a motor needs a driver, not a pin.

A **for loop** repeats a block a fixed number of times. It has three parts inside its brackets: where to start, how long to keep going, and what to change each time round.

The shape of a for loop

```
for (int i = 0; i < 5; i++) {   // start at 0, stop below 5, add 1 each time
```

```
  Serial.println(i);            // prints 0, 1, 2, 3, 4
}
```


Far more important this year is the **function** - an instruction you write yourself. Taking five lines of measuring code and giving them one clear name turns an unreadable loop into something a stranger could follow. Professional programmers do this constantly, and from Unit 3 onwards so will you.

The shape of a function

```
int readDistance() {        // int means: this hands back a whole number
  // ... the measuring steps go in here ...
  return answer;            // return means: hand this value back
}

void loop() {
  int d = readDistance();   // one clear line instead of five confusing ones
}
```

5.1 Why a Motor Cannot Touch a Pin

An Arduino output pin is designed to signal, not to supply power. It can deliver about 20 milliamps safely. A BO motor starting under load can demand several hundred milliamps - more than ten times as much. Connect one directly and the pin does not simply fail; it burns out, and the damage is permanent.

The solution is a **motor driver**. Think of it as a relay race. The Arduino sends a tiny signal saying *turn this motor forwards*. The driver, connected to a battery, does the heavy lifting. The Arduino never touches the motor current at all - and that is exactly the point.

Did You Know

The same idea scales all the way up. A driver in an electric car takes a signal from a small computer and switches hundreds of amps to the wheels. Your L298N and that system are solving exactly the same problem, just at different sizes.

New word	What it means
Microsecond	One millionth of a second.
pulseIn()	Measures how long a pin stays HIGH, in microseconds.
Function	An instruction you write yourself and then reuse.
return	Hands a value back from a function to whoever called it.
Motor driver	A circuit that switches heavy current on a small signal's command.